

Digital Systems and Computer Architecture

Complete Study Notes - Units 3, 4, and 5

Course: Digital Systems and Computer Architecture
Institution: RV University
Instructor: Mouli Sankaran

Overview and Learning Objectives

This comprehensive guide covers three fundamental units in Digital Systems and Computer Architecture: Sequential Circuits (Unit 3), Memory Systems (Unit 4), and CPU Design and Instruction Cycles (Unit 5). These notes provide complete coverage of sequential machines, computer evolution, CPU architecture, memory hierarchies, and basic CPU design principles. By mastering this material, you will understand how sequential circuits store and process information over time, how modern computers evolved to their current state, and how CPUs are designed. This knowledge forms the foundation for understanding computer organization and digital system design.

UNIT 3: SEQUENTIAL CIRCUITS

Module 3.1: Introduction to Sequential Circuits

3.1.1 What are Sequential Circuits?

Definition: Sequential circuits are combinational circuits with feedback elements that store internal states (0 or 1), where outputs depend on both current inputs and previous internal states.

Key Characteristics:

- **Memory Capability:** Store binary information in storage elements
- **State-Dependent:** Output = $f(\text{current inputs, present state})$
- **Time-Dependent:** Behavior described by time sequences of inputs, outputs, and states
- **Feedback Loops:** Include feedback paths to maintain state information

Comparison with Combinational Circuits:

Feature	Combinational Circuits	Sequential Circuits
Memory	No memory elements	Has memory elements
Output depends on	Current inputs only	Current inputs + Past states
Feedback	No feedback	Contains feedback
Examples	Adders, Multiplexers, Decoders	Counters, Registers, State Machines
Speed	Generally faster	Generally slower

State Definition: The binary information stored in storage elements at any given time defines the state of the sequential circuit at that time.

Next State Function: The next state of storage elements is a function of:

1. External inputs
2. Present state

3.1.2 Classification of Sequential Circuits

Sequential circuits are classified based on the timing of their signals:

A. Asynchronous Sequential Circuits

Characteristics:

- Feedback/memory elements built with combinational circuits
- **No clock signal** to synchronize activities
- State changes occur due to propagation delays in feedback elements
- Faster operation than synchronous circuits
- Prone to **unstable states** and **race conditions**
- Rarely used in modern designs due to stability issues

Race Condition Example:

Consider inputs x_1 and x_2 both set to 1:

- Output Y will oscillate between 1 and 0 indefinitely
- Circuit becomes unstable
- No predictable final state

Why Rarely Used:

1. Difficult to design reliably
2. Timing analysis is complex
3. Unpredictable behavior in edge cases
4. Hard to debug and maintain

B. Synchronous Sequential Circuits

Characteristics:

- Memory elements are **clocked flip-flops**
- State transitions occur **only at clock edges**
- State is "frozen" between clock transitions
- Each state maintained for exactly one clock period
- Stable and predictable operation
- Industry standard for digital design

Advantages:

- Predictable timing behavior
- Easier to design and debug
- Reliable state transitions
- Compatible with standard design tools

Clock Signal Properties:

- Periodic square wave
- Defines when state changes occur
- Synchronizes all operations in the circuit

Module 3.2: Storage Elements - Latches and Flip-Flops

3.2.1 Fundamental Concepts

Storage Element Definition: A digital circuit component that can maintain a binary state (0 or 1) indefinitely as long as power is supplied by an input signal.

Two Types of Storage Elements:

Feature	Latch	Flip-Flop
Trigger Type	Level-triggered	Edge-triggered
Sensitivity	Responds to signal levels	Responds to signal transitions
Stability	Transparent when enabled	Changes only at clock edges
Clock Required	No (or level-based)	Yes (edge-based)
Response	Continuous when enabled	Discrete at clock edges
Usage	Less common in modern designs	Building blocks of sequential circuits
Construction	Built from gates	Built from latches
Examples	SR Latch, D Latch	D Flip-Flop, JK Flip-Flop, T Flip-Flop
Classification	None specific	Asynchronous or Synchronous

3.2.2 Level Triggering vs Edge Triggering

Level Triggering

Mechanism: Based on recognizing a specified signal level (HIGH or LOW)

Characteristics:

- Output remains in triggered condition as long as input is at trigger level
- Continuous monitoring of input signal
- Used in data acquisition and control systems
- **Cannot regulate precise timing**
- More susceptible to noise

Applications:

- Data acquisition systems
- Control systems requiring continuous monitoring
- Simple enable/disable circuits

Edge Triggering

Mechanism: Based on detecting a sharp transition (edge) in the input signal

Types of Edges:

1. **Positive Edge (Rising Edge):** Transition from 0 to 1
2. **Negative Edge (Falling Edge):** Transition from 1 to 0

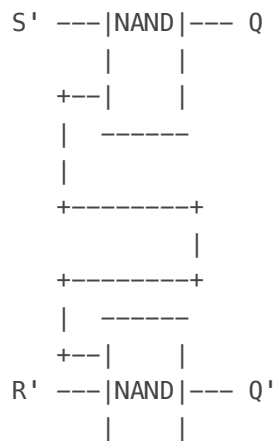
Characteristics:

- Output changes state when trigger edge is detected
- Changes to opposing state on edge detection
- **Precise timing control**
- Used in synchronous circuits (counters, flip-flops)
- Immune to level noise

Applications:

- Counters
- Registers
- State machines
- Any circuit requiring precise timing

3.2.3 SR Latch (Set-Reset Latch)

Implementation with NAND Gates (S'R' Latch):**Truth Table (Active LOW inputs):**

S'	R'	Q	Q'	State
0	0	∞	∞	FORBIDDEN
0	1	1	0	SET
1	0	0	1	RESET
1	1	Q_prev	Q'_prev	HOLD/MEMORY

Operation:

- **SET (S'=0, R'=1):** Sets Q to 1, Q' to 0
- **RESET (S'=1, R'=0):** Resets Q to 0, Q' to 1
- **HOLD (S'=1, R'=1):** Maintains previous state
- **FORBIDDEN (S'=0, R'=0):** Both outputs try to be 1, creates indeterminate next state

Why S'=0, R'=0 is Forbidden:

- Both Q and Q' would be forced to 1
- Violates complementary relationship
- When inputs return to 1,1 - unpredictable race condition
- Final state depends on which gate responds first

Key Terminology:

- **SET:** Setting a value to ONE (1)
- **RESET:** Resetting a value to ZERO (0)

3.2.4 SR Flip-Flop (Clocked SR Latch)

Structure: SR Latch with added clock control

Operation:

- Inputs (S and R) are gated with clock signal (C)
- State changes only when clock is active (level-triggered) or transitions (edge-triggered)
- Provides synchronization capability

Truth Table (Clocked SR):

C	S	R	Q(next)	Action
0	X	X	Q	No change (clock disabled)
1	0	0	Q	Hold
1	0	1	0	Reset
1	1	0	1	Set
1	1	1	∞	Forbidden

Module 3.3: D Flip-Flop

3.3.1 D Flip-Flop Basics

Definition: A Data (D) flip-flop is the simplest type of flip-flop that captures the value on the D input and transfers it to the output

Key Feature: Eliminates the forbidden state problem of SR flip-flops

Operation:

- **D = 0:** Q becomes 0 at next clock edge
- **D = 1:** Q becomes 1 at next clock edge
- Q follows D, but changes only at clock edges

Truth Table:

D	Q(next)
0	0
1	1

Characteristic Equation: $Q(\text{next}) = D$

3.3.2 Positive Edge-Triggered D Flip-Flop

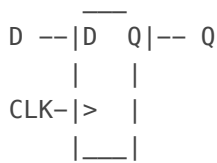
Master-Slave Configuration:

- Consists of two latches in series
- **Master latch:** Captures input before clock edge
- **Slave latch:** Transfers to output at clock edge

Operation:

1. Before positive edge: Master captures D value
2. At positive edge: Slave copies master's value to Q
3. Output Q shows value that was on D immediately before positive edge

Logic Symbol:



The triangle (>) indicates positive edge triggering

Advantages:

- Simple design with minimal gates
- No invalid states
- Predictable operation
- Foundation for other flip-flop types

3.3.3 Applications of D Flip-Flops

A. Data Storage (Registers)

4-Bit Register:

- Four D flip-flops in parallel
- Common clock signal
- Stores 4-bit data word
- Data captured simultaneously on clock edge

Usage:

- Temporary data storage
- Pipeline stages
- Buffering

B. Serial to Parallel Converter

Configuration:

- Chain of D flip-flops
- Output of one connects to input of next
- Common clock signal
- Serial data shifts through chain

Operation:

- 1. Serial data enters first flip-flop
- 2. Each clock pulse shifts data one position
- 3. After n clock pulses, parallel data available at all outputs

Example: 4-bit converter

- After 4 clock pulses: $Q_3Q_2Q_1Q_0$ contains parallel data
- Used in serial communication interfaces

C. Frequency Division

Configuration:

- Connect Q' (inverted output) back to D input
- Creates toggle behavior

Operation:

- Output frequency = Input clock frequency / 2
- Can cascade for further division

Module 3.4: JK Flip-Flop

3.4.1 JK Flip-Flop Basics

Definition: A JK flip-flop is an improved version of the SR flip-flop that eliminates the forbidden state by introducing a toggle func

Named After: Jack Kilby, inventor of the integrated circuit (IC) in 1958, Nobel Prize in Physics (2000)

Truth Table:

J	K	Q(next)	Action
0	0	Q	Hold (No change)
0	1	0	Reset
1	0	1	Set
1	1	Q'	Toggle

Characteristic Equation: $Q(\text{next}) = JQ' + K'Q$

Key Innovation: When $J=1$ and $K=1$, instead of forbidden state, the flip-flop **toggles** (Q becomes Q')

3.4.2 JK Flip-Flop Implementation Using D Flip-Flop

Logic:

- $D = JQ' + K'Q$
- Use combinational logic to generate D input
- Use D flip-flop for storage

Advantages:

- Most versatile flip-flop
- Can implement SR, D, and T flip-flop functions
- No invalid states

Applications:

- Counters
- State machines
- Control circuits

3.4.3 JK Flip-Flop Waveform Analysis

Given: Positive edge-triggered JK flip-flop, initially RESET (Q=0)

Analysis Steps:

1. Identify each positive clock edge
2. Check J and K values just before edge
3. Determine Q(next) from truth table
4. Draw resulting waveform

Common Patterns:

- J=1, K=0: Output goes HIGH and stays
 - J=0, K=1: Output goes LOW and stays
 - J=1, K=1: Output toggles at each clock edge
 - J=0, K=0: Output maintains current state
-

Module 3.5: T Flip-Flop

3.5.1 T Flip-Flop Basics

Definition: A Toggle (T) flip-flop changes state (toggles) when T=1 and holds state when T=0.

Truth Table:

T	Q(next)	Action
0	Q	Hold
1	Q'	Toggle

Characteristic Equation: $Q(\text{next}) = TQ' + T'Q = T \oplus Q$

Implementation from JK:

- Connect J and K together
- T input drives both J and K
- When T=0: J=K=0 → Hold
- When T=1: J=K=1 → Toggle

Implementation from D:

- $D = T \oplus Q$
- Requires XOR gate

3.5.2 Applications of T Flip-Flop

A. Counters

- Natural choice for binary counters
- Each flip-flop divides frequency by 2
- Toggle function creates counting sequence

B. Frequency Division

- Connect T to 1 (always toggle)
 - Output frequency = Input frequency / 2
-

Module 3.6: Finite State Machines (FSM)

3.6.1 FSM Fundamentals

Definition: A Finite State Machine (FSM), also called Finite Automaton, is a mathematical model of computation that can be in exactly one state at any given time.

Components of FSM:

1. **States:** Finite set of conditions the machine can be in
2. **Initial State:** Starting state of the machine
3. **Inputs:** External signals that trigger transitions
4. **Transitions:** Rules for moving between states
5. **Outputs:** Signals produced by the machine

FSM Characteristics:

- Can be in exactly **one state** at any time
- Changes from one state to another in response to inputs
- Transitions are well-defined
- Number of states is finite

3.6.2 FSM Example: Turnstile

Scenario: Controlled access gate that allows one person per coin

States:

1. **LOCKED** (initial state)
2. **UNLOCKED**

Inputs:

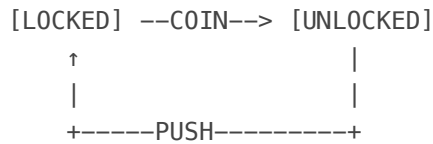
1. **COIN:** Coin is dropped
2. **PUSH:** Person pushes the bar

State Transitions:

- LOCKED + COIN → UNLOCKED
- UNLOCKED + PUSH → LOCKED
- LOCKED + PUSH → LOCKED (no effect)

- UNLOCKED + COIN → UNLOCKED (wasted coin)

State Diagram:



Dark dot at LOCKED indicates initial state

Operation:

1. System starts in LOCKED state
2. Dropping coin transitions to UNLOCKED
3. Pushing bar in UNLOCKED state allows one person through and returns to LOCKED
4. Pushing in LOCKED state has no effect
5. Ensures exactly one person per coin

3.6.3 Moore vs Mealy State Machines

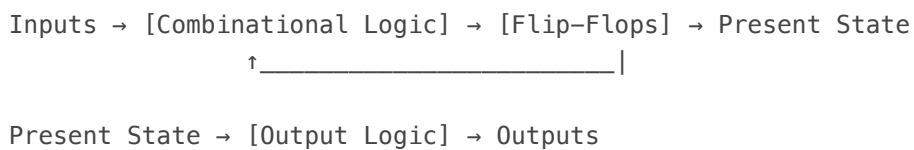
Moore Model

Definition: Output is a function of **present state only**

Characteristics:

- Output depends solely on current state
- Output changes only when state changes
- Output is synchronized with clock
- Generally requires more states
- More stable outputs (no glitches)

Block Diagram:



When to Use:

- When output stability is critical
- When outputs should change synchronously
- In control applications

Mealy Model

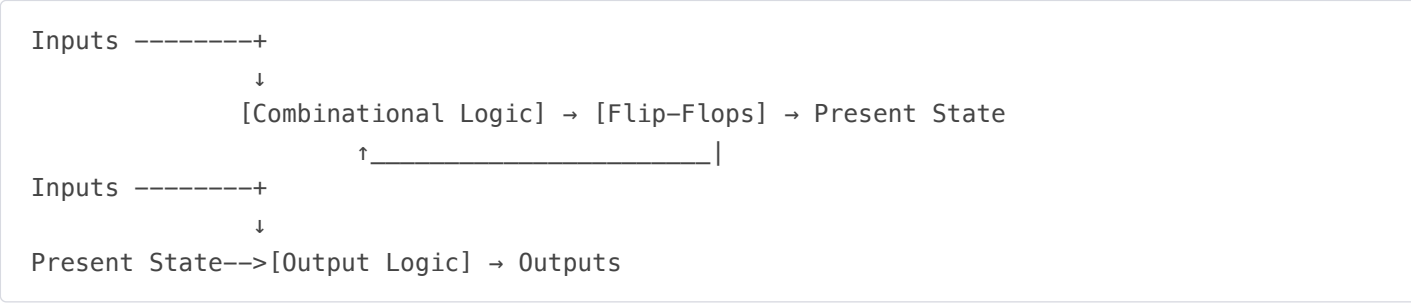
Definition: Output is a function of **present state AND inputs**

Characteristics:

- Output depends on both state and current inputs
- Output can change immediately when inputs change
- Faster response to inputs
- Generally requires fewer states

- May have output glitches

Block Diagram:



When to Use:

- When fast response is needed
- When fewer states are desired
- In applications where transient outputs are acceptable

Comparison Table:

Feature	Moore Machine	Mealy Machine
Output depends on	State only	State + Inputs
Output timing	Changes with state	Can change anytime
Number of states	Generally more	Generally fewer
Output stability	More stable	May have glitches
Response time	Slower	Faster
Design complexity	Simpler output logic	More complex output logic
Synchronization	Fully synchronized	Partially asynchronous

Note: A circuit can have both Moore and Mealy type outputs simultaneously.

3.6.4 FSM Analysis Steps

To analyze a sequential circuit, follow these four steps:

1. Logic/Circuit Diagram

- Draw or examine the actual circuit
- Identify flip-flops, gates, and connections
- Note input and output signals

2. State Equations

- Write equations for next state of each flip-flop
- Express in terms of present state and inputs
- Example: $Q(\text{next}) = D = f(Q, \text{inputs})$

3. State Table

- List all possible combinations of present states and inputs
- Calculate next state for each combination
- Calculate outputs for each combination

4. State Diagram

- Visual representation of state transitions
- Circles represent states
- Arrows represent transitions with conditions
- Outputs labeled on states (Moore) or transitions (Mealy)

Unit 3 Summary: Key Takeaways

1. **Sequential circuits** have memory and outputs depend on both inputs and previous states
2. **Synchronous circuits** use clocked flip-flops; **Asynchronous circuits** use combinational feedback (unstable)
3. **Latches** are level-triggered; **Flip-flops** are edge-triggered
4. **D flip-flop** is simplest, directly transfers input to output at clock edge
5. **JK flip-flop** eliminates forbidden state with toggle function ($J=K=1$)
6. **T flip-flop** toggles when $T=1$, holds when $T=0$
7. **Moore machines** have outputs based only on state; **Mealy machines** have outputs based on state and inputs
8. **FSM analysis** involves: circuit $\rightarrow$ equations $\rightarrow$ table $\rightarrow$ diagram

Unit 3: Top 10 Most Testable Facts

1. Sequential circuits differ from combinational by having **memory/feedback** and **time-dependent behavior**
2. SR latch forbidden state: **$S'=0, R'=0$** creates indeterminate next state
3. D flip-flop characteristic equation: **$Q(\text{next}) = D$**
4. JK flip-flop characteristic equation: **$Q(\text{next}) = JQ' + K'Q$**
5. T flip-flop characteristic equation: **$Q(\text{next}) = T \oplus Q$**
6. **Positive edge triggering** (rising edge: $0 \rightarrow 1$) vs **Negative edge triggering** (falling edge: $1 \rightarrow 0$)
7. Moore machine: outputs = **$f(\text{state only})$** ; Mealy machine: outputs = **$f(\text{state} + \text{inputs})$**
8. JK flip-flop named after **Jack Kilby**, inventor of IC
9. Serial-to-parallel converter requires **n clock pulses** for n -bit conversion
10. Asynchronous circuits are **faster but unstable**; Synchronous circuits are **stable but slower**

Unit 3: Practice Questions

Multiple Choice Questions

Q1. What is the main difference between combinational and sequential circuits?

- A) Sequential circuits are faster
- B) Sequential circuits have memory elements
- C) Combinational circuits use flip-flops
- D) Combinational circuits need clock signals

Answer: B

Explanation: Sequential circuits contain memory elements (flip-flops/latches) that store state, while combinational circuits have no inputs.

Q2. In an SR latch with NAND gates, which input combination is forbidden?

- A) $S'=0, R'=0$

- B) $S'=1, R'=1$
- C) $S'=0, R'=1$
- D) $S'=1, R'=0$

Answer: A

Explanation: When both S' and R' are 0, both outputs try to be 1, violating the complementary relationship and creating an indeterminate state.

Q3. A JK flip-flop with $J=1, K=1$ will:

- A) Set the output to 1
- B) Reset the output to 0
- C) Toggle the output
- D) Hold the previous state

Answer: C

Explanation: When $J=K=1$, the JK flip-flop toggles, meaning Q becomes Q' .

Q4. Which flip-flop is best suited for frequency division?

- A) SR flip-flop
- B) D flip-flop
- C) T flip-flop
- D) Latch

Answer: C

Explanation: T flip-flop with $T=1$ toggles at every clock edge, dividing frequency by 2.

Q5. In a Moore machine, the output depends on:

- A) Current inputs only
- B) Present state only
- C) Present state and current inputs
- D) Previous state only

Answer: B

Explanation: Moore machine outputs are a function of the present state only.

Short Answer Questions

Q6. Explain why asynchronous sequential circuits are rarely used despite being faster.

Answer: Asynchronous circuits are rarely used because they suffer from instability and race conditions due to lack of clock synchronisation. In feedback paths, these can cause unpredictable oscillations. While they are faster, the difficulty in design, analysis, and debugging makes them unsuitable for most applications.

Q7. What is the difference between level-triggered and edge-triggered devices?

Answer: Level-triggered devices (latches) respond to signal levels and are transparent when enabled - they continuously monitor inputs. Edge-triggered devices (flip-flops) respond only to signal transitions (edges) and change state only at specific clock edges (rising or falling), providing precise timing immunity.

Q8. Draw the truth table for a T flip-flop and explain its operation.

Answer:

T	Q(next)
0	Q
1	Q'

When $T=0$, the flip-flop holds its current state. When $T=1$, the flip-flop toggles (Q becomes Q'). This makes it ideal for counters and

Q9. How many clock pulses are required for a 4-bit serial-to-parallel converter to have complete parallel data available?

Answer: Four clock pulses are required. Serial data is shifted into the first flip-flop at each positive clock edge, and after four pulses $Q_3Q_2Q_1Q_0$.

Q10. What are the advantages of a Mealy machine over a Moore machine?

Answer:

1. **Fewer states** - generally requires less states for same functionality
2. **Faster response** - outputs can change immediately with input changes, not waiting for next clock
3. **More compact** - fewer hardware resources needed

However, Mealy machines may have output glitches and are more complex to design.

UNIT 4: COMPUTER ARCHITECTURE AND MEMORY SYSTEMS

Module 4.1: Evolution of Computers

4.1.1 Computer Generations

First Generation (1946-1959): Vacuum Tubes

ENIAC (Electronic Numerical Integrator And Computer)

- Built at University of Pennsylvania, USA (1946)
- World's first general-purpose electronic computer
- **Specifications:**
 - Weight: 30 tonnes (30,000 kg)
 - Area: 1,500 square feet
 - Vacuum tubes: 18,000
 - Power consumption: 140 kilowatts
 - Speed: 5,000 additions per second

Technology: Vacuum tubes (thermionic valves)

Characteristics:

- Large physical size
- High power consumption
- Generated excessive heat
- Unreliable (frequent tube failures)
- Very expensive
- Slow by modern standards

Second Generation (1959-1965): Transistors

Key Innovation: Invention of transistor by Bell Labs (1947)

Improvements over First Generation:

- Smaller size
- Lower power consumption
- More reliable
- Faster operation
- Less heat generation
- Lower cost

Third Generation (1965-1971): Integrated Circuits (ICs)

Technology: LSI (Large Scale Integration)

- Multiple transistors on single chip
- Miniaturization continues

Benefits:

- Even smaller size
- Much faster
- More reliable
- Significantly cheaper
- Lower power consumption

Fourth Generation (1971-Present): VLSI

Technology: VLSI (Very Large Scale Integration)

- Millions/billions of transistors per chip
- Microprocessors

Modern Era:

- Extremely compact
- Very high speed
- Highly reliable
- Affordable
- Energy efficient (relatively)

Trend Summary:

Generation	Technology	Speed	Power	Size	Cost
1st	Vacuum Tubes	↓	↑	↑	↑
2nd	Transistors	↑	↓	↓	↓
3rd	IC (LSI)	↑↑	↓↓	↓↓	↓↓
4th	VLSI	↑↑↑	↓↓↓	↓↓↓	↓↓↓

4.1.2 Moore's Law

Definition: The observation that the number of transistors per square inch on integrated circuits doubles approximately every 18 m

Origin:

- Made by Gordon Moore, Intel co-founder (1965)
- Originally observed yearly doubling
- Revised to 18-month doubling cycle

Implications:

1. Exponential growth in computing power
2. Decreasing cost per transistor
3. Smaller feature sizes
4. More functionality per chip

Current Status:

- Law still roughly holds but slowing
- Physical and economic limits approaching
- Transition to 3D structures and new materials

Graph Insight: Logarithmic plot shows exponential growth from 1970s to present

4.1.3 Technology Nodes

Definition: The smallest feature size possible in a semiconductor manufacturing process, measured in nanometers (nm).

Examples:

- **22 nm node:** Smallest circuit feature is 22 nanometers thick
- **10 nm node:** Features are 10 nanometers
- **3 nm node:** Current cutting-edge technology (design stage)

Silicon Wafer Manufacturing:**Process:**

1. Design circuits
2. Create photolithographic masks
3. Etch circuits onto silicon wafer (17-inch diameter)
4. Test on-wafer
5. Cut into individual chips (dies)
6. Bond copper wires to connect signals
7. Package with protective casing and pins

Yield Consideration:

- Not all chips on wafer are functional
- Smaller nodes = more defects
- Each square on wafer is individual chip

4.1.4 Semiconductor Foundries

Foundry (Fab): Manufacturing facility where integrated circuits are produced

Major Foundries:

Rank	Company	Location	Market Share
1	TSMC (Taiwan Semiconductor Manufacturing Company)	Taiwan	~60%
2	Samsung Foundry	South Korea	~18%
3	UMC (United Microelectronics Corp)	Taiwan	~7%
4	SMIC (Semiconductor Manufacturing International Corp)	China (Shanghai)	~5%

Companies with Own Fabs (IDM - Integrated Device Manufacturers):

- Intel
- Samsung
- Texas Instruments (TI)
- STMicroelectronics
- NXP
- Maxim
- Micron
- SanDisk
- Cypress
- SK Hynix
- Analog Devices

Fabless Companies (Design Only):

- **Apple** - Designs A-series chips (e.g., A10 Fusion with 3.3 billion transistors, 64-bit ARMv8)
- Qualcomm
- Broadcom
- AMD
- MediaTek
- NVIDIA
- Marvell

Example: iPhone 7/7 Plus used A10 Fusion chip

- Designed by Apple
- Manufactured by TSMC
- 4 cores, 3.3 billion transistors
- 64-bit ARMv8 architecture

Important Manufacturing Video:

The course references a video on IC manufacturing that may be tested!

Module 4.2: CPU Clocks and Performance

4.2.1 Clock Signal Generation

Crystal Oscillator:

Principle: Piezoelectric effect

- Apply voltage → crystal deforms (mechanical stress)
- Crystal vibrates → generates voltage oscillations
- Creates precise electrical signal with fixed frequency

Components:

- Quartz crystal (primary)
- LC tank circuit (symbol representation)

Frequency Range: Typically up to hundreds of MHz (200–400 MHz maximum for quartz)

Why Quartz?

- Highly stable frequency
- Temperature resistant
- Low cost
- Reliable over time

4.2.2 GHz Clock Generation

Challenge: Quartz crystals cannot directly generate GHz frequencies

Solution: Phase-Locked Loop (PLL)

PLL Function:

- Electronic circuit with feedback
- Multiplies input frequency
- Example: 100 MHz crystal → 3 GHz output (30× multiplication)

Process:

Crystal (100 MHz) → PLL (×30) → 3 GHz CPU Clock

4.2.3 Clock Speed and Performance

Clock Speed Definition: Number of cycles per second (measured in Hz)

Relationship:

- Period (T) = 1 / Frequency (f)
- Frequency (f) = 1 / Period (T)

Example Comparison:

Processor	Clock Speed	Period per Cycle	Time for ADD Instruction (1 cycle)
Processor 1	2 MHz	0.5 μs	0.5 μs
Processor 2	1 MHz	1.0 μs	1.0 μs

Processor 1 is 2× faster for same instruction

Analogy: Lower clock speed = watching video in slow motion

4.2.4 CPU Speed Limits

Historical Milestone:

- First 1 GHz processor: Year 2000
- **World Record:** 8.805 GHz (AMD FX-8150, 2011, overclocked with liquid nitrogen)
- **Commercial Limit:** ~4 GHz maximum

Why Can't We Go Higher?

1. Heat Generation (Primary Limitation)

- High frequency → faster transistor switching
- Faster switching → more power dissipation
- Power dissipation $\propto$ frequency $\times$ voltage²
- At high speeds, cooling becomes impossible
- 8+ GHz requires liquid nitrogen cooling

2. Power Density

- Modern chips: ~10 nm technology nodes
- Higher transistor density
- More heat in smaller area
- Cannot efficiently remove heat

3. Physical Limits

- Signal propagation delays
- Quantum effects at small scales
- Leakage currents

Solution: Multi-Core Processors

- Instead of 1 core at 8 GHz
- Use 4-8 cores at 3-4 GHz each
- Better performance with manageable heat
- Parallel processing

4.2.5 Overclocking

Definition: Running CPU at higher frequency than rated specification

Requirements:

- Increase voltage (Vcore)
- Enhanced cooling
- Motherboard support

Example: Intel Core i7

Mode	Frequency	Vcore	Notes
Normal	3.6 GHz	1.20 V	Standard operation
Turbo Boost	~3.8 GHz	Auto	Dynamic, built-in
Overclock	4.0 GHz	1.28 V	Manual, more heat

Risks:

- Increased heat
- Reduced lifespan
- Potential instability
- Voided warranty

Turbo Boost: Intel's dynamic overclocking for graphics/compute bursts

Module 4.3: CPU vs Memory Performance Gap

4.3.1 The Memory Wall

Problem: CPU speeds have increased much faster than memory speeds

Historical Trend:

- CPU performance: Exponential growth
- Memory performance: Linear/slow growth
- Gap widening over decades

Consequences:

1. **Memory Latency:** CPU often waits for data from memory
2. **Performance Bottleneck:** Memory access is limiting factor
3. **CPU Underutilization:** Processor idle during memory access

4.3.2 Solution: Memory Hierarchy

Cache Memory System:

- Small, fast memory between CPU and main memory
- Holds frequently accessed data
- Multiple levels (L1, L2, L3)

Hierarchy (Fastest to Slowest):

1. **CPU Registers** - Fastest, smallest
2. **L1 Cache** - Very fast, small (32-64 KB per core)
3. **L2 Cache** - Fast, medium (256 KB - 512 KB per core)
4. **L3 Cache** - Moderately fast, larger (2-32 MB shared)
5. **Main Memory (RAM)** - Slower, large (GB)
6. **Storage (SSD/HDD)** - Slowest, largest (TB)

Cache Hit: Data found in cache (fast access)

Cache Miss: Data not in cache, must fetch from memory (slow)

Goal: Keep CPU fed with data to minimize idle time

Module 4.4: I/O Evolution

4.4.1 I/O Speed Progression

Note: bps vs Bps

- **bps:** bits per second (lowercase 'b')
- **Bps:** Bytes per second (uppercase 'B')
- 1 Bps = 8 bps
- 1 Kb/s = 1000 bits/s
- 1 KB/s = 1024 bytes/s

Historical I/O Speeds:

(Refer to document figure for specific progression)

Modern High-Speed Interfaces:

- **PCIe 3.0 x16:** ~16 GB/s
 - **USB 3.0:** 5 Gb/s (~625 MB/s)
 - **SATA III:** 6 Gb/s (~600 MB/s)
 - **NVMe SSD:** 3-7 GB/s
-

Module 4.5: Motherboard Architecture

4.5.1 Intel Hub Architecture

Major Components:**1. CPU (Central Processing Unit)**

- Main processor
- Mounted in socket on motherboard
- 1366 pins (Intel Core i7 example)
- Heat generation: 100-140W
- Requires heatsink and fan cooling

2. North Bridge (Memory Controller Hub)

- Connects CPU to high-speed components
- Interfaces:
 - **FSB (Front Side Bus):** CPU communication
 - **RAM:** Main memory interface
 - **PCIe (PCI Express):** High-speed I/O (graphics cards)

3. South Bridge (I/O Controller Hub)

- Connects low-speed peripherals
- Interfaces:
 - **USB:** Universal Serial Bus (keyboards, mice, storage)
 - **SATA/IDE:** Hard disk interfaces
 - **PCI:** Peripheral Component Interconnect (legacy expansion cards)
 - **Legacy ports:** Serial, parallel (RS-232)
 - **Audio:** Sound card interface

4. RAM (Random Access Memory)

- **SO-DIMM:** Small Outline Dual In-line Memory Module
- **DDR2/DDR3/DDR4 SDRAM:** Double Data Rate Synchronous DRAM
- 200-pin modules (laptop), 240-pin (desktop)
- Example capacity: 2 GB, 4 GB, 8 GB per module

5. Expansion Slots:

- **PCIe x16:** Graphics cards (highest bandwidth)
- **PCIe x1, x4:** Other expansion cards
- **PCI:** Legacy cards

6. Storage:

- Hard disk drives
- Solid state drives (SSDs)
- Optical drives

7. Power Supply:

- Converts AC (~230V) to DC voltages
- Provides: +12V, +5V, +3.3V, -12V, -5V
- Sends "Power Good" signal to motherboard
- Typical power: 400-1000W

4.5.2 Heatsink and Cooling

Why Needed?

- High-performance chips generate 100+ Watts
- Example: Intel Core i7-5930K at 3.3 GHz → 140W

Heatsink Construction:

- Material: Aluminum or copper
- Fins increase surface area
- Fan forces airflow
- Thermal paste improves contact

Cooling Methods:

1. Air cooling (most common)
2. Liquid cooling (high-performance)
3. Passive cooling (low-power systems)

Module 4.6: Computer Boot Process

4.6.1 Software Layers

Hierarchy (Low to High):

```
Hardware
  ↓
Firmware (BIOS/UEFI)
  ↓
Bootloader
  ↓
Operating System Kernel
  ↓
System Software
  ↓
Application Software
```

4.6.2 Detailed Boot Sequence

Step 1: Power On

Process:

1. AC power applied to power supply

2. Power supply converts AC → DC voltages
3. Power supply performs self-test
4. "Power Good" signal sent to motherboard
5. Motherboard generates RESET signal to CPU

CPU Response to RESET:

- All registers cleared to known state
- **Program Counter (PC)** set to reset vector address
 - Typical addresses: 0xF000_0000 or 0x0000_0000
- CPU ready to fetch first instruction

Step 2: BIOS Execution

BIOS (Basic Input/Output System):

- **Location:** ROM chip on motherboard (~16 MB)
- **Type:** Firmware (low-level software with hardware access)
- **Language:** Written in C or Assembly
- **Independence:** OS-independent
- **Manufacturer:** Supplied by motherboard vendor

POST (Power-On Self-Test):

1. Component Detection:

- Check for presence of components
- Keyboard, display, CD drive, etc.

2. Memory Test:

- Test RAM functionality
- Determine available memory range
- Initialize memory controller

3. Device Initialization:

- Initialize display adapter
- Initialize storage controllers
- Set up basic I/O

4. Boot Device Selection:

- Check BIOS configuration
- Determine boot order (CD, HDD, Network, USB)

Error Handling:

- Beep codes for hardware failures
- Display error messages

Step 3: CMOS Configuration

CMOS (Complementary Metal Oxide Semiconductor) Memory:

- **Size:** ~256 bytes
- **Power:** Dedicated CMOS battery (maintains data when PC is off)
- **Contents:**
 - BIOS settings
 - Boot sequence configuration
 - Date and time

- Hardware parameters

Real-Time Clock (RTC):

- Powered by same CMOS battery
- Maintains time even when computer is off
- Updates date/time in CMOS

Accessing BIOS Setup:

- Press DEL, F2, or F12 during boot
- Modify boot order, time, hardware settings

Step 4: Bootloader Execution

Process:

1. BIOS reads **boot sector** from selected device
2. Boot sector contains **bootloader** program
3. Bootloader loaded into RAM
4. Bootloader takes control

Bootloader Functions:

1. Locate operating system on disk
2. Load OS kernel into RAM
3. Load essential OS data structures
4. Transfer control to OS kernel

Common Bootloaders:

- **GRUB:** Linux systems
- **Windows Boot Manager:** Windows
- **U-Boot:** Embedded systems

Step 5: Operating System Loading

OS Kernel Loading:

1. Bootloader loads kernel image to specific RAM address
2. Loads initial RAM disk (initrd) if needed
3. Sets up memory management
4. Initializes kernel data structures

OS Takes Control:

- Initializes device drivers
- Mounts file systems
- Starts system services
- Loads user interface
- System ready for user

4.6.3 Modern UEFI

UEFI (Unified Extensible Firmware Interface):

- Replacement for legacy BIOS
- More features and flexibility

Advantages over BIOS:

1. **Security:** Secure Boot, cryptography support
2. **Diagnostics:** Remote diagnostics and repair
3. **GUI:** Graphical interface with mouse support
4. **Larger Drives:** Supports >2 TB disks (GPT)
5. **Faster Boot:** Optimized boot process
6. **Networking:** Built-in network stack

Current Status: Most modern computers (post-2010) use UEFI

Module 4.7: Program Architecture Models

4.7.1 von Neumann Architecture

Characteristics:

- **Single memory** for both program and data
- **Sequential execution** of instructions
- **Shared bus** for code and data access
- Instructions executed one after another

Structure:

```
    CPU
    |
  [Control Unit]
    |
  [Bus Interface]
    |
  ===BUS===
    |
  [Memory]
  (Code + Data)
```

Operation:

1. Control unit fetches instruction from memory
2. Decodes instruction
3. Executes instruction
4. May fetch data from same memory
5. Repeat

Limitation: von Neumann Bottleneck

- Code and data share same bus
- Cannot fetch instruction and data simultaneously
- CPU may be idle waiting for memory access

Advantages:

- Simpler design
- Flexible (code and data in same space)
- Stored-program concept

Disadvantages:

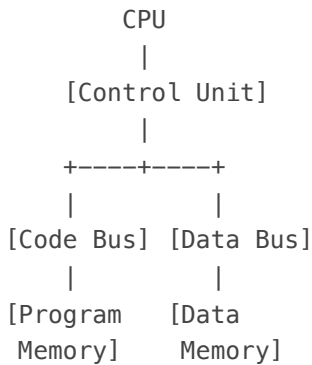
- Slower (sequential access)
- Memory bottleneck

4.7.2 Harvard Architecture

Characteristics:

- **Separate memories** for program and data
- **Parallel access** capability
- **Dedicated buses** for code and data

Structure:



Operation:

1. Fetch instruction from program memory
2. **Simultaneously** fetch data from data memory
3. Execute instruction
4. Parallel operation continues

Advantages:

1. **Higher Performance:** Parallel access
2. **No bottleneck:** Separate paths
3. **Optimized:** Different memory technologies for code/data

Modern Implementation:

- Most modern CPUs use **Modified Harvard Architecture**
- **Internal:** Separate L1 instruction and data caches
- **External:** Unified memory (von Neumann externally)
- Best of both worlds

Examples:

- ARM processors (separate I-cache and D-cache)
- Intel/AMD processors (split L1 cache)
- DSP processors
- Microcontrollers

4.7.3 Cache and Harvard Architecture

Modern CPU Cache Organization:

- **L1 Cache:** Split into:
 - **L1-I:** Instruction cache

- **L1-D:** Data cache
- Separate paths to CPU core
- Implements Harvard architecture internally

Benefits:

- CPU can fetch instruction and data in same cycle
- Eliminates von Neumann bottleneck at cache level
- Maintains compatibility with unified external memory

Unit 4 Summary: Key Takeaways

1. **Computer evolution:** Vacuum tubes → Transistors → ICs → VLSI, each generation smaller, faster, cheaper
 2. **Moore's Law:** Transistor count doubles every ~18 months
 3. **Technology nodes:** Measure smallest feature size (currently 3-5 nm)
 4. **TSMC** is world's largest foundry (~60% market share)
 5. **CPU clock:** Generated by quartz crystal + PLL for GHz frequencies
 6. **CPU speed limit:** ~4 GHz due to heat; solution is multi-core
 7. **Memory wall:** CPU much faster than memory; solved with cache hierarchy
 8. **Motherboard:** North Bridge (fast I/O) + South Bridge (slow I/O)
 9. **Boot sequence:** Power → BIOS → POST → CMOS → Bootloader → OS
 10. **Harvard architecture:** Separate code/data memories; modern CPUs use internally with split L1 cache
-

Unit 4: Top 10 Most Testable Facts

1. **ENIAC specifications:** 30 tonnes, 1500 sq ft, 18,000 vacuum tubes, 140 kW power
 2. **Moore's Law:** Transistor count doubles every **18 months**
 3. **CPU speed record:** **8.805 GHz** (AMD FX-8150, 2011, liquid nitrogen cooling)
 4. **Commercial CPU limit:** ~4 GHz due to heat generation issues
 5. **TSMC market share:** ~60% (world's largest foundry)
 6. **PLL:** Multiplies crystal frequency to achieve GHz clock speeds
 7. **Intel Core i7 pins:** **1366** pins
 8. **BIOS ROM size:** ~16 MB
 9. **CMOS memory size:** ~256 bytes
 10. **Harvard architecture advantage:** **Parallel** code and data access
-

Unit 4: Practice Questions

Multiple Choice Questions

Q1. Which generation of computers used vacuum tubes?

- A) Second generation
- B) First generation
- C) Third generation
- D) Fourth generation

Answer: B

Explanation: First generation computers (1946-1959) used vacuum tubes. ENIAC is the classic example.

Q2. According to Moore's Law, transistor count doubles every:

- A) 12 months
- B) 18 months
- C) 24 months
- D) 6 months

Answer: B

Explanation: Moore's Law states transistor count doubles approximately every 18 months (originally observed as yearly, revised to 18 months).

Q3. What is the primary reason CPU clock speeds haven't increased beyond ~4 GHz commercially?

- A) Lack of technology
- B) Cost concerns
- C) Heat generation/cooling limitations
- D) Software limitations

Answer: C

Explanation: At high frequencies, transistor switching generates excessive heat that cannot be efficiently removed, making high clock speeds impractical due to cooling requirements.

Q4. Which component connects the CPU to high-speed peripherals like graphics cards?

- A) South Bridge
- B) North Bridge
- C) BIOS
- D) CMOS

Answer: B

Explanation: North Bridge (Memory Controller Hub) connects CPU to high-speed components including RAM and PCIe slots for graphics cards.

Q5. What does BIOS stand for?

- A) Basic Instruction Operating System
- B) Binary Input Output System
- C) Basic Input Output System
- D) Boot Initialization Operating System

Answer: C

Explanation: BIOS stands for Basic Input Output System, firmware that initializes hardware during boot.

Q6. The Harvard architecture differs from von Neumann by having:

- A) Faster CPU
- B) Separate code and data memories
- C) More registers
- D) Better graphics

Answer: B

Explanation: Harvard architecture has separate physical memories for program and data, allowing parallel access.

Short Answer Questions

Q7. Why do modern CPUs use multi-core designs instead of increasing single-core clock speed?

Answer: Multi-core designs are used because single-core clock speeds are limited by heat generation. Running one core at 8 GHz (with liquid nitrogen), whereas 4 cores at 3 GHz each provide better total performance with manageable heat dissipation. Multi-core also enables parallel processing of tasks.

throughput.

Q8. Explain the "memory wall" problem and its solution.

Answer: The memory wall refers to the growing performance gap between CPUs and memory. CPU speeds have increased exponentially, while memory access speeds have grown slowly, creating a bottleneck where the CPU often waits for data. The solution is a memory hierarchy with multiple levels of cache. Cache provides fast access to frequently used data, minimizing CPU idle time.

Q9. What is the purpose of CMOS memory in a computer?

Answer: CMOS memory (256 bytes) stores BIOS configuration settings including boot device order, hardware parameters, and date/time. It is powered by a dedicated battery so settings persist when the computer is turned off. The battery also powers the Real-Time Clock (RTC) to maintain system time.

Q10. Describe the complete boot sequence from power-on to OS loading.

Answer:

1. **Power On:** AC power converted to DC by power supply, which sends "Power Good" signal to motherboard
2. **CPU Reset:** Motherboard generates RESET signal; CPU clears registers and sets Program Counter to reset vector (0xF000_0000)
3. **BIOS Execution:** CPU runs firmware from ROM, performs POST (Power-On Self-Test) - checks components, tests memory, initializes hardware
4. **Configuration:** Reads CMOS memory for boot order and settings
5. **Bootloader:** BIOS loads boot sector from selected device, bootloader program executes
6. **OS Loading:** Bootloader loads OS kernel into RAM and transfers control to operating system

UNIT 5: CPU DESIGN AND INSTRUCTION CYCLE

Module 5.1: Digital Logic Basics for CPU Design

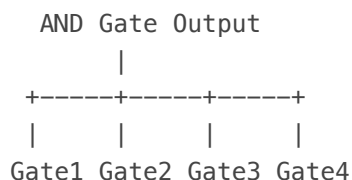
5.1.1 Fan-Out and Fan-In

Fan-Out

Definition: The number of gate inputs that a single gate output can drive without degradation of logic levels.

Measurement: Count how many device inputs are connected to one output

Example:



Fan-out = 4 (AND gate drives 4 other gates)

Electrical Consideration:

- Each gate input draws current
- If output current is I_{out} mA and fan-out is 4
- Each input receives $I_{out}/4$ mA
- Excessive fan-out → insufficient current → logic level degradation

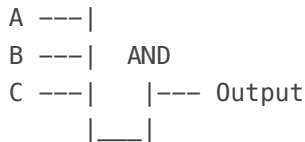
Practical Implications:

- TTL gates: typical fan-out = 10
- CMOS gates: very high fan-out (50+)
- Must check fan-out specifications in datasheets
- Use buffers if fan-out exceeds limit

Fan-In

Definition: The number of inputs a gate can handle

Example:



Fan-in = 3 (3-input AND gate)

Performance Consideration:

- **Large fan-in → Slower gate operation**
- More inputs → more internal transistors
- Longer propagation delay
- More complex internal structure

Design Trade-off:

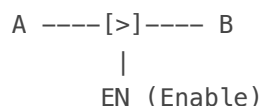
- Large fan-in: Fewer gates needed
- Small fan-in: Faster operation
- Often use tree structures for high fan-in functions

5.1.2 Tri-State Buffers (3-State Logic)

Definition: A buffer that can output three states: Logic 0, Logic 1, or High-Impedance (Z)

Three States:

1. **Logic 0 (LOW):** Output driven to 0
2. **Logic 1 (HIGH):** Output driven to 1
3. **High-Impedance (Z):** Output disconnected (open circuit)

Symbol and Operation:**Truth Table:**

EN (Enable)	A	B (Output)	Physical Connection
0	X	Z (High-Z)	Open circuit (A disconnected from B)
1	0	0	Short circuit (A connected to B)
1	1	1	Short circuit (A connected to B)

Active HIGH Control: EN=1 enables connection; EN=0 disconnects

High-Impedance (Z) State:

- Output effectively disconnected from input
- Appears as open circuit
- Does not drive the line
- Allows other devices to drive the same line

Critical Application: Bus Systems

- Multiple devices share one bus
- Only one device drives at a time (others in high-Z)
- Prevents conflicts and short circuits
- Essential for CPU bus architecture

Bus Conflict Prevention:

```
Device 1 --[Tri-state]--+
                        |
Device 2 --[Tri-state]--+---- Bus Line
                        |
Device 3 --[Tri-state]--+
```

Rules:

- Only ONE device enabled at a time
- Others must be in high-Z state
- Prevents multiple drivers

Module 5.2: Register Design and Bus Architecture

5.2.1 8-bit Register Using D Flip-Flops

Structure:

- **8 D flip-flops** (one per bit)
- **Common clock (CP):** All flip-flops share same clock input
- **Parallel Input:** 8 input lines (D₇-D₀)
- **Parallel Output:** 8 output lines (Q₇-Q₀)

Operation:

1. Input data present on D₇-D₀ lines
2. On positive clock edge (CP)
3. All 8 bits latched simultaneously into flip-flops
4. Outputs Q₇-Q₀ now reflect inputs
5. Data held until next clock edge

8-bit D-type Data Latch (74LS374):

- Standard IC implementing 8-bit register
- Industry-standard component
- Used extensively in CPU design

Key Features:

- Synchronous operation (all bits update together)
- Edge-triggered (positive edge)
- Stores 8-bit data word
- Foundation for larger registers (16-bit, 32-bit, 64-bit)

5.2.2 Connecting Two Registers

Problem: Direct connection requires dedicated buses

Simple Two-Register System:

```
Register A (32-bit)
  ↓ 32 lines
  ↓
Register B (32-bit)
```

Bidirectional Connection Requires:

- 32 lines from A to B
- 32 lines from B to A
- Total: **64 interconnections** for just 2 registers!

Control Signals Needed:

- Output enable for Register A
- Input enable for Register B
- Output enable for Register B
- Input enable for Register A

Scaling Problem:

- 3 registers: Exponentially more connections
- N registers: $N(N-1)$ buses needed
- Quickly becomes unmanageable

5.2.3 Need for a Common Bus

Problem with Individual Connections:

- Processors have 20-30 registers (modern: 100+)
- Individual connections = exponential interconnects
- Impractical routing
- Huge silicon area
- Complex control logic

Solution: Common Bus Architecture

- All registers connect to **one shared bus**
- Dramatically reduces interconnections
- Only ONE register drives bus at a time

- Multiple registers can read simultaneously

Benefits:

1. **Minimal Interconnections:** Linear growth instead of exponential
2. **Simplified Routing:** Single set of wires
3. **Scalable:** Easy to add more registers
4. **Area Efficient:** Less silicon real estate
5. **Standard Architecture:** Used in all modern CPUs

5.2.4 Register Bus Architecture

N-bit Bus Configuration:

```

Register 0 ----[Tri-state Buffer]-----+
                                           |
Register 1 ----[Tri-state Buffer]-----+
                                           |-- N-bit Bus
Register 2 ----[Tri-state Buffer]-----+
                                           |
Register N ----[Tri-state Buffer]-----+

```

Control Signals:

- **Output Enable (OE):** Allows register to drive bus
- **Input Enable (IE) / Write Enable (WE):** Allows register to capture from bus

Key Rules:

1. **Only ONE output enabled at a time** (prevents bus conflict)
2. **Multiple inputs can be enabled** (broadcast data to multiple registers)
3. **All disabled outputs must be in high-Z state**
4. **Bus notation /n:** indicates n-bit wide bus (e.g., /32 = 32-bit bus)

Typical Operations:

- **Read:** Enable one register's output, disable all others
- **Write:** Enable target register's input, one or more sources
- **Transfer:** R0 → R1: Enable R0 output, enable R1 input

5.2.5 Bus Operation Example: RN + Tmp = R0

Scenario: Add contents of Register N and Temp Register, store result in R0

Components:

- Registers: R0, R1, ..., RN
- Temp Register (Tmp)
- ALU (Arithmetic Logic Unit)
- Bus D (Data bus)

Operation Sequence:

Step 1: RN → Tmp

- Enable RN output (drive Bus D)
- Enable Tmp input
- Disable all other outputs and inputs

- Clock pulse → Tmp = RN

Step 2: Provide Operands to ALU

- Enable RN output (one ALU operand)
- Enable Tmp output (second ALU operand)
- Both values on bus? No! ALU has dedicated inputs
- ALU reads both registers

Step 3: ALU Performs Addition

- ALU configured for ADD operation
- Result = RN + Tmp
- ALU output appears on Bus D

Step 4: Store Result in R0

- Enable ALU output to Bus D
- Enable R0 input only
- Disable all other register inputs
- Clock pulse → R0 = ALU result

Control Timing:

Each step occurs in **one clock cycle**

Key Insight:

- ALU output drives bus when enabled
- All bus lines carry ALU result
- Only enabled inputs capture the data
- Protects other registers from unwanted writes

Module 5.3: Bus Control Sequences - Practice

5.3.1 Quiz 1: R0 = RN + RN

Problem: Calculate $R0 = RN + RN$ (double the value in RN)

Given Choices:

- A. Enable output of RN and input of Temp Reg (Temp = RN)
- B. Enable output of RN and output of Temp Reg (ALU operands)
- C. Give ADD command operation to ALU
- D. Enable input of only R0 to store the result ($R0 = RN + RN$)

Correct Sequence: B, C, D, A

Explanation:

Wait, let me reconsider. To compute $RN + RN$:

1. First load RN into Temp: **A**
2. Then provide both RN and Temp to ALU: **B**
3. Command ALU to ADD: **C**
4. Store result in R0: **D**

Correct Answer: A, B, C, D

Each step happens in one clock cycle.

5.3.2 Quiz 2: $R0 = 2 \times RN$

Problem: Calculate $R0 = 2 \times RN$ using ALU

Choices:

- A. Enable output of RN and input of Temp Reg (Temp = RN)
- B. Enable output of RN and output of Temp Reg (ALU operands)
- C. Give ADD command operation to ALU
- D. Enable input of only R0 to store result ($R0 = RN + RN$)

Solution: Multiplication by 2 = Addition of number to itself

- $2 \times RN = RN + RN$

Correct Sequence: A, B, C, D

1. Copy RN to Temp
2. Provide RN and Temp to ALU
3. ADD operation
4. Store in R0

5.3.3 Quiz 3: $R0 = 0$ (Using ALU)

Problem: Clear R0 to zero using ALU operations

Choices:

- A. Move R0 to Temp Register
- B. Give R0 as another operand to ALU
- C. Give XOR command operation to ALU
- D. Enable R0 input to take in the output of ALU ($R0 = 0$)

Solution: Use property: $X \oplus X = 0$ (XOR of any value with itself is 0)

Correct Sequence: A, B, C, D

1. Temp = R0
2. ALU inputs: R0 and Temp (same value)
3. XOR operation: $R0 \oplus R0 = 0$
4. Store result (0) back in R0

Why Use XOR?

- XOR truth table: $0 \oplus 0 = 0$, $1 \oplus 1 = 0$
- Any number XOR itself = 0
- Common trick for clearing registers

Module 5.4: Basic Instruction Cycle

5.4.1 CPU Instruction Execution Model

Sequential Execution:

- CPU executes assembly instructions one after another
- Each instruction goes through defined steps
- Continuous cycle as long as clock runs

Basic Instruction Cycle Definition:

Execution of one complete assembly instruction

Example Instruction:

```
ADD R1, R2, [R3] ; R1 = R2 + value at memory address in R3
```

This uses **indexed addressing mode**

5.4.2 Steps in Basic Instruction Cycle

Step 1: Instruction Fetch (IF)

- Fetch instruction from memory
- Use Program Counter (PC) to get address
- Transfer instruction to Instruction Register (IR)
- Increment PC for next instruction

Step 2: Instruction Decode (ID)

- Decode the operation (ADD, SUB, LOAD, etc.)
- Identify source and destination registers
- Determine addressing mode
- Control unit generates appropriate control signals

Step 3: Operand Fetch / Data Fetch (OF/DF)

- Fetch operands from registers or memory
- For memory operands, compute effective address
- Example: [R3] means use R3's value as memory address
- Retrieve data from computed address

Step 4: Execute (EX)

- Perform the operation in ALU
- Example: Addition, subtraction, logical operation
- May involve address computation for memory access
- Result produced

Step 5: Write Back / Store Result (WB)

- Store result in destination register or memory
- Update flags if necessary
- Complete the instruction

Cycle Repeats:

- Each instruction follows these steps
- CPU continues until halt or interrupt

5.4.3 ALU Dual Role

1. Data Operations:

- Arithmetic: ADD, SUB, MUL, DIV
- Logical: AND, OR, XOR, NOT
- Shift/Rotate operations

2. Address Computations:

- Calculate effective addresses
- Example: **MOV R1, [R2 + R3]**
 - ALU computes: $R2 + R3 = \text{effective address}$
 - Then memory access at that address
- Relative addressing: $PC + \text{offset}$
- Indexed addressing: $\text{base} + \text{index}$

Why ALU for Addresses?

- Addresses are just numbers
 - Use same arithmetic hardware
 - Efficient resource utilization
 - Single ALU serves both purposes
-

Module 5.5: CPU Architecture Overview

5.5.1 Components of CPU

1. Datapath

Includes:

- **Registers:** Store data and addresses
- **ALU:** Performs operations
- **Internal Buses:** Connect components
- **Multiplexers:** Route data
- **Temporary Storage:** Hold intermediate results

2. Control Unit

Functions:

- **Instruction Decode:** Interpret instructions
- **Control Signal Generation:** Activate appropriate datapath elements
- **Sequencing:** Coordinate instruction steps
- **Timing:** Synchronize operations with clock

CPU = Datapath + Control Unit

5.5.2 Datapath Details

Definition: The collection of functional units that perform data processing operations

Components:

Registers:

- General-purpose registers ($R0, R1, \dots, RN$)
- Special-purpose registers:
 - **PC (Program Counter):** Holds address of next instruction
 - **IR (Instruction Register):** Holds current instruction
 - **MAR (Memory Address Register):** Holds memory address for access
 - **MDR (Memory Data Register):** Holds data being transferred to/from memory
 - **Status Registers:** Flags (Zero, Carry, Sign, Overflow)

ALU:

- Arithmetic operations
- Logical operations
- Comparison operations
- Shift operations

Internal Buses:

- Data bus
- Address bus
- Control lines

Data Flow:

- Data transfers along datapath during program execution
- Controlled by control unit signals
- Represents Register Transfer Level (RTL) description

5.5.3 Control Unit Architecture

Centralized vs Distributed Control:**Centralized Control:**

- Single control unit for entire CPU
- Simple small processors

Distributed Control:

- Multiple control units
- **CPU Control Unit:** Instruction execution
- **Memory Controller:** Memory operations
- **I/O Controllers:** Device-specific control
- Used in complex systems

Why Distributed?

- Complex systems too large for single controller
- Each subsystem has specialized control needs
- Parallel operation
- Modularity

Modern Implementation:

- Hierarchical control
- Main CPU controller coordinates
- Subsystem controllers handle details
- Communication through well-defined interfaces

5.5.4 Register Unit Design

Multi-Port Register File:**Requirements:**

- Multiple simultaneous accesses
- Typical: **2 Read Ports + 1 Write Port**

2 Read Ports:

- Allows reading two operands simultaneously
- Essential for binary operations (ADD, SUB, etc.)
- Example: ADD R3, R1, R2 needs to read R1 and R2 together

1 Write Port:

- Allows writing result back
- Can occur in same cycle as reads
- Example: Write to R3 while next instruction reads R1, R2

Implementation:

- Tri-state buffers for outputs
- Multiplexers for input selection
- Separate address decoders for each port
- Shared storage cells

Why Multi-Port?

- Performance: Reduce instruction cycle time
 - Parallelism: Fetch operands together
 - Pipelining: Enable overlapped execution
-

Module 5.6: Instruction Fetch Example

5.6.1 General Purpose Register (GPR) Based CPU

Architecture:

- Multiple general-purpose registers (R0-RN)
- Registers can hold data or addresses
- Flexible instruction set
- Register-to-register operations are fast

Contrast with Accumulator Architecture:

- Accumulator: One special register for operations
- GPR: Any register can be source or destination
- GPR more flexible and efficient

5.6.2 Instruction Fetch Sequence

Initial State:

- PC contains address of next instruction
- Example: PC = 0x1000

Fetch Steps:

1. $MAR \leftarrow PC$

- Transfer PC value to Memory Address Register
- MAR now holds 0x1000

2. Initiate Memory Read

- Send read signal to memory

- Address 0x1000 on address bus

3. Wait for Memory

- Memory access takes time (latency)
- CPU waits for memory ready signal

4. MDR ← Memory[MAR]

- Instruction data from memory arrives
- Loaded into Memory Data Register

5. IR ← MDR

- Transfer instruction to Instruction Register
- IR now holds the instruction to be decoded

6. PC ← PC + 4

- Increment PC to next instruction
- Assumes 4-byte instructions (32-bit architecture)
- PC now = 0x1004

Ready for Decode:

- Instruction in IR ready to be decoded
- PC pointing to next instruction
- Fetch cycle complete

Timing:

- Each step takes one or more clock cycles
- Memory access is slowest part
- May use cache to speed up

Unit 5 Summary: Key Takeaways

1. **Fan-out:** Number of inputs one output can drive; **Fan-in:** Number of inputs a gate has
2. **Tri-state buffers:** Enable sharing buses by disconnecting (high-Z) when not active
3. **Common bus architecture:** All registers share one bus; only one output active at a time
4. **Register file:** Typically has **2 read ports and 1 write port** for parallel operand access
5. **Basic instruction cycle:** Fetch → Decode → Operand Fetch → Execute → Write Back
6. **ALU dual role:** Data operations AND address calculations
7. **CPU = Datapath + Control Unit**
8. **Datapath:** Registers, ALU, buses; **Control Unit:** Sequencing and control signals
9. **Distributed control:** Modern systems use multiple controllers (CPU, memory, I/O)
10. **Instruction fetch:** PC → MAR → Memory → MDR → IR, then increment PC

Unit 5: Top 10 Most Testable Facts

1. **Tri-state buffer states:** Logic 0, Logic 1, and **High-Impedance (Z)**
2. **High-Z state:** Output is **disconnected** (open circuit)
3. **Bus rule:** Only **ONE** device can drive the bus at a time
4. **74LS374:** Standard **8-bit D-type data latch** IC
5. **Register file ports:** Typically **2 read ports + 1 write port**
6. **Instruction cycle phases:** **IF-ID-OF-EX-WB** (Fetch-Decode-Operand Fetch-Execute-Write Back)

7. **PC increment:** For 32-bit architecture, PC increments by **4** (bytes) after each instruction
 8. **XOR clear trick:** $R \oplus R = 0$ (used to clear registers)
 9. **Fan-in effect:** Large fan-in $\rightarrow$ **slower** gate operation
 10. **CPU components:** CPU = **Datapath + Control Unit**
-

Unit 5: Practice Questions

Multiple Choice Questions

Q1. What is the high-impedance (Z) state in tri-state logic?

- A) Logic level 0
- B) Logic level 1
- C) Output disconnected (open circuit)
- D) Output shorted to ground

Answer: C

Explanation: High-impedance state means the output is effectively disconnected, appearing as an open circuit and not driving the bus.

Q2. In a common bus architecture, how many devices can drive the bus simultaneously?

- A) As many as needed
- B) Only one
- C) Two maximum
- D) All connected devices

Answer: B

Explanation: Only ONE device can drive the bus at any time to prevent conflicts. All others must be in high-Z state.

Q3. A typical register file in a CPU has:

- A) 1 read port, 1 write port
- B) 2 read ports, 1 write port
- C) 1 read port, 2 write ports
- D) 2 read ports, 2 write ports

Answer: B

Explanation: Standard register files have 2 read ports (for two operands) and 1 write port (for result), enabling efficient binary operations.

Q4. What is the correct order of the basic instruction cycle phases?

- A) Execute $\rightarrow$ Fetch $\rightarrow$ Decode $\rightarrow$ Write Back
- B) Fetch $\rightarrow$ Execute $\rightarrow$ Decode $\rightarrow$ Write Back
- C) Fetch $\rightarrow$ Decode $\rightarrow$ Operand Fetch $\rightarrow$ Execute $\rightarrow$ Write Back
- D) Decode $\rightarrow$ Fetch $\rightarrow$ Execute $\rightarrow$ Write Back

Answer: C

Explanation: The instruction cycle follows: IF (Fetch) $\rightarrow$ ID (Decode) $\rightarrow$ OF (Operand Fetch) $\rightarrow$ EX (Execute) $\rightarrow$ WB (Write Back).

Q5. For a 32-bit instruction architecture, by how much does the Program Counter (PC) increment after fetching one instruction?

- A) 1 byte
- B) 2 bytes
- C) 4 bytes
- D) 8 bytes

Answer: C

Explanation: 32-bit = 4 bytes, so PC increments by 4 to point to the next instruction.

Q6. Which operation can be used to clear a register to zero using the ALU?

- A) AND the register with itself
- B) OR the register with itself
- C) XOR the register with itself
- D) ADD the register to itself

Answer: C

Explanation: XOR operation with same value always produces 0 ($R \oplus R = 0$), commonly used to clear registers.

Q7. The datapath of a CPU includes all of the following EXCEPT:

- A) ALU
- B) Registers
- C) Instruction decode logic
- D) Internal buses

Answer: C

Explanation: Instruction decode logic is part of the Control Unit, not the Datapath. Datapath includes ALU, registers, and buses.

Q8. What is fan-out in digital circuits?

- A) Number of inputs a gate has
- B) Number of gate inputs that one output can drive
- C) Number of gates in a circuit
- D) Speed of signal propagation

Answer: B

Explanation: Fan-out is the number of gate inputs that a single gate output can drive without signal degradation.

Short Answer Questions

Q9. Explain why a common bus architecture is preferred over individual register-to-register connections in a CPU.

Answer:

A common bus architecture is preferred because:

1. **Scalability:** N registers require only one N-bit bus instead of $N(N-1)$ individual buses
2. **Area Efficiency:** Dramatically reduces silicon real estate and routing complexity
3. **Simplicity:** Easier to manage and control one shared resource
4. **Cost:** Fewer interconnections mean lower manufacturing cost

For example, 30 registers would need 870 individual buses (30×29) versus just 1 shared bus.

Q10. Describe the complete sequence of control operations needed to perform $R0 = R1 + R2$ using a bus-based architecture.

Answer:

Step 1 (Cycle 1): $Temp \leftarrow R1$

- Enable R1 output
- Enable Temp input
- Clock pulse

Step 2 (Cycle 2): ALU inputs $\leftarrow$ R1, Temp

- Enable R1 output (first operand)
- Enable Temp output (second operand)
- Both feed into ALU

Step 3 (Same as Step 2): ALU performs addition

- Send ADD command to ALU
- ALU computes $R1 + \text{Temp}$

Step 4 (Cycle 3): $R0 \leftarrow$ ALU result

- Enable ALU output to bus
- Enable R0 input only
- Clock pulse
- R0 now contains $R1 + R2$

Actually, more efficiently:

- Step 1:** $\text{Temp} \leftarrow R1$
- Step 2:** Provide R2 and Temp to ALU
- Step 3:** Configure ALU for ADD
- Step 4:** $R0 \leftarrow$ ALU output

This completes in approximately 3-4 clock cycles depending on architecture.

Comprehensive Unit Review: Quick Reference Tables

Sequential Circuit Types Comparison

Type	Clock	Triggering	Stability	Usage	Speed
Asynchronous	No	Propagation delay	Unstable	Rare	Fast
Synchronous	Yes	Clock edge	Stable	Common	Moderate

Flip-Flop Comparison Chart

Type	Inputs	Forbidden State?	Toggle?	Characteristic Equation	Primary Use
SR	S, R	Yes ($S=R=1$)	No	$Q+ = S + R'Q$	Basic storage
D	D	No	No	$Q+ = D$	Data storage, registers
JK	J, K	No	Yes ($J=K=1$)	$Q+ = JQ' + K'Q$	Universal, counters
T	T	No	Yes ($T=1$)	$Q+ = T \oplus Q$	Counters, frequency division

Computer Generation Evolution

Gen	Years	Technology	Size	Speed	Power	Cost	Example
1st	1946-59	Vacuum Tubes	Huge	Very Slow	Very High	Very High	ENIAC
2nd	1959-65	Transistors	Large	Slow	High	High	IBM 7094
3rd	1965-71	IC (LSI)	Medium	Moderate	Medium	Medium	IBM 360
4th	1971-Now	VLSI	Small	Very Fast	Low	Low	Modern PCs

Memory Hierarchy (Fastest → Slowest)

Level	Size	Speed	Cost/Bit	Technology
Registers	Bytes	Fastest	Highest	Flip-flops
L1 Cache	32-64 KB	Very Fast	Very High	SRAM
L2 Cache	256-512 KB	Fast	High	SRAM
L3 Cache	2-32 MB	Moderate	Medium	SRAM
Main Memory	GB	Slow	Low	DRAM
SSD	TB	Very Slow	Very Low	Flash
HDD	TB	Slowest	Lowest	Magnetic

Bus Control Operations Summary

Operation	Sequence	Clock Cycles
R0 = RN	Enable RN out, Enable R0 in	1
RN + Tmp → R0	Tmp=RN, Setup ALU, ADD, R0=result	3-4
R0 = 2×RN	Tmp=RN, RN+Tmp, R0=result	3-4
R0 = 0 (XOR)	Tmp=R0, XOR, R0=result	3-4

Common Question Patterns with Answers

Pattern 1: "What is the difference between X and Y?"

- Examples: Latch vs Flip-flop, Moore vs Mealy, Asynchronous vs Synchronous
- **Answer Strategy:** Create comparison table, list 3-5 key differences, give examples

Pattern 2: "Draw the waveform/timing diagram"

- Examples: D flip-flop output, JK flip-flop toggle
- **Answer Strategy:** Mark clock edges, apply truth table at each edge, show output transitions

Pattern 3: "Give the sequence of control signals"

- Examples: Register transfer operations

- **Answer Strategy:** List steps chronologically, specify enable signals, note bus usage

Pattern 4: "Why is [technology/method] used instead of [alternative]?"

- Examples: Multi-core vs high-frequency, Common bus vs individual connections
- **Answer Strategy:** List practical limitations of alternative, benefits of chosen approach

Pattern 5: "Explain the boot sequence / instruction cycle"

- **Answer Strategy:** Number the steps, show data flow, specify component involvement

One-Page Cheat Sheet

UNIT 3: Sequential Circuits

- **Sequential = Combinational + Memory**
- **Latch:** Level-triggered | **Flip-Flop:** Edge-triggered
- **SR:** $S=R=1$ forbidden | **D:** $Q+=D$ | **JK:** $Q+=JQ'+K'Q$, $J=K=1$ toggles | **T:** $Q+=T\oplus Q$
- **Moore:** $\text{Output}=f(\text{state})$ | **Mealy:** $\text{Output}=f(\text{state}, \text{inputs})$

UNIT 4: Computer Architecture

- **Generations:** Vacuum $\rightarrow$ Transistor $\rightarrow$ IC $\rightarrow$ VLSI
- **Moore's Law:** $2\times$ transistors every 18 months
- **CPU Limit:** ~ 4 GHz (heat), solution=multi-core
- **Boot:** Power $\rightarrow$ BIOS(POST) $\rightarrow$ CMOS $\rightarrow$ Bootloader $\rightarrow$ OS
- **Harvard:** Separate code/data memory (modern CPUs use internally)

UNIT 5: CPU Design

- **Fan-out:** # inputs one output drives | **Fan-in:** # inputs gate has
- **Tri-state:** 0, 1, Z (high-impedance)
- **Bus Rule:** Only 1 output active at a time
- **Register File:** 2 read + 1 write ports
- **Instruction Cycle:** IF $\rightarrow$ ID $\rightarrow$ OF $\rightarrow$ EX $\rightarrow$ WB
- **CPU:** Datapath (registers+ALU+buses) + Control Unit

Key Equations

- $T = 1/f$ (period = $1/\text{frequency}$)
- $\text{Power} \propto f \times V^2$ (frequency $\times$ voltage²)
- XOR clear: $R\oplus R = 0$
- PC increment (32-bit) = +4 bytes

Critical Numbers

- ENIAC: 30 tonnes, 18K tubes, 140 kW
- TSMC: $\sim 60\%$ foundry market share
- CPU record: 8.805 GHz (liquid N₂)
- Core i7: 1366 pins, 140W

- CMOS: 256 bytes
 - BIOS: ~16 MB
-

Study Plan Recommendation

For 1-Week Preparation:

- **Day 1-2:** Unit 3 (Flip-flops, FSM) - Practice waveform analysis
- **Day 3-4:** Unit 4 (Evolution, CPU speed, Boot) - Memorize key numbers
- **Day 5-6:** Unit 5 (Registers, buses, instruction cycle) - Practice control sequences
- **Day 7:** Review cheat sheet, practice MCQs, quick revision

For 3-Day Preparation:

- **Day 1:** All units - Read through once, highlight key points
- **Day 2:** Practice questions, waveform problems, control sequences
- **Day 3:** Cheat sheet memorization, MCQ practice, Top 10 facts review

Last-Minute (1 Day):

- Focus on: Top 10 Most Testable Facts (all units)
 - Review: Comparison tables, cheat sheet
 - Practice: 10-15 MCQs from each unit
 - Memorize: Critical numbers (ENIAC, Moore's Law, CPU limits, etc.)
-

References

Primary Source:

- DSCA_UNIT3.pdf - Sequential Circuits (Mouli Sankaran, RVU)
- DS_CA_UNIT-5.pdf - CPU Design and Instruction Cycles (Mouli Sankaran, RVU)
- DS_CA_UNIT-4.pdf - Computer Architecture and Memory Systems (Mouli Sankaran, RVU)

Additional Resources:

- Digital Design - Morris Mano (Sequential circuits, flip-flops)
 - Computer Organization and Architecture - William Stallings (CPU design)
 - IC Manufacturing Video (referenced in Unit 4)
-

End of Comprehensive Study Notes

Total Coverage:

- 3 Complete Units
- 15+ Modules
- 50+ Subtopics
- 30+ Practice Questions

- Multiple Comparison Tables
- Quick Reference Cheat Sheet
- Study Plans for Different Timeframes

These notes are designed to be your ONLY resource for exam preparation. Good luck!